Programming tools for Mathematics

Chapter 2: Vectors and Matrices

Dr. Lakehal Soumaya

HIS - Algiers

Academic Year 2025–2026

Chapter Overview

1. Vectors
   - Description
   - Adressing and indexing
   - Operations on vectors
   - Special functions
   - Practice Exercises
2. Matrices
   - General form
   - Matrix creation
   - Indexing and Access
   - The size of a matrix
   - Matrix Operations
   - Matrix functions
   - Training Exercises

Vectors

MATLAB was originally designed to allow mathematicians, scientists, and engineers to easily use linear algebra mechanisms. We first present vectors and then matrices, which are very intuitive and convenient in MATLAB.

1.1 Description

A vector is an ordered list of elements. If the elements are arranged horizontally, it is called a row vector; if they are arranged vertically, it is called a column vector.
To create a row vector, simply write the list of its components between brackets [ ] and separate them with spaces or commas as follows:

```
>> v = [1 3.4 5 -6]
v =
1.0000 3.4000 5.0000 -6.0000
```

1.1 Description (continued)

To create a column vector, write the list of its components between brackets [ ] and separate them with semicolons:

```
>> U = [4 ; -2 ; 1]
U =
4 -2 1
```

Or compute the transpose of a row vector:

```
>> U = [4 -2 1]'
U =
4 -2 1
```

1.1 Description (continued)

If the components of a vector X are ordered with consecutive values and a step (increment/decrement) different from 1, we can specify the step as follows:

```
X = [initial_value :  increment :  final_value]
```

Example:

```
>> x = [0 :  pi/10 :  pi]
x =
Columns 1 through 7 0 0.3142 0.6283 0.9425 1.2566 1.5708
1.8850
Columns 8 through 11 2.1991 2.5133 2.8274 3.1416
```

1.1 Description (continued)

More complex expressions can also be written:

```
>> V = [1:2:5 , -2:2:1]

V =

1  3  5 -2  0

>> A = [1 2 3]

A =

1 2 3

>> B = [A, 4, 5, 6]

B =
```

1.2 Addressing and Indexing

Access to vector elements is done using the general syntax:

```
vector_name(positions)
```

where positions can be a single index or a list of indices.

- Parentheses ( ) are used for access
- Brackets [ ] are used only during creation

Example with:

```
V = [5, -1, 13, -6, 7]
```

1.2 Addressing and Indexing (Examples)

```
>> V(3)          % 3rd position
ans = 13

>> V(2:4)        % from 2nd to 4th
ans = -1 13 -6

>> V(4:-2:1)     % from 4th to 1st with step -2
ans = -6 -1

>> V(3:end)      % from 3rd to last
ans = 13 -6 7

>> V([1,3,4])    % positions 1, 3, 4
ans = 5 13 -6
```

1.2 Addressing and Indexing (Modification)

```
>> V(1) = 8
V = 8 -1 13 -6 7

>> V(6) = -3
V = 8 -1 13 -6 7 -3

>> V(9) = 5
V = 8 -1 13 -6 7 -3 0 0 5

>> V(2) = []
V = 8 13 -6 7 -3 0 0 5

>> V(3:5) = []
V = 8 13 0 0 5
```

1.3 Operations on Vectors

Given two vectors:

```
>> u = [-2, 6, 1];
>> v = [3, -1, 4];
```

Addition:

```
>> u + 2
ans = 0 8 3

>> u + v
ans = 1 5 5
```

Subtraction:

```
>> u - 2
ans = -4 4 -1

>> u - v
ans = -5 7 -3
```

1.3 Operations on Vectors (continued)

Element-wise multiplication:

```
>> u .* 2
ans = -4 12 2

>> u .* v
ans = -6 -6 4
```

Element-wise division:

```
>> u ./ 2
ans = -1.0000 3.0000 0.5000

>> u ./ v
ans = -0.6667 -6.0000 0.2500
```

Element-wise power:

```
>> u.^2
ans = 4 36 1
```

1.4 Special Functions

Some commonly used vector functions:

- `linspace(start, end, n)`: creates a vector with evenly spaced elements
- `sum(V)`: sum of elements
- `prod(V)`: product of elements
- `mean(V)`: average of elements
- `sort(V)`: sorts elements in ascending order
- `length(V)`: size of the vector
- `max(V)`: largest element
- `min(V)`: smallest element
- `fliplr(V)`: reverses the order of elements

1.5 Practice Exercises

1. Given two vectors $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$, write a program that produces a vector $z = (z_1, z_2, \ldots, z_n)$ where each component z_i is the value (with the largest absolute value) between x_i and y_i.
2. Create a row vector starting at 1 and ending at 33 with increment 2:

$$(1, 3, 5, \ldots, 33)$$

3. Create a column vector starting at 15, decreasing with increment -5, and ending at -25. (Hint: use transpose if needed.)

2.1 Matrices

A matrix is a two-dimensional array of elements.
Vectors are matrices with a single row or a single column.
To define a matrix in MATLAB, the following rules must be respected:

- Elements must be enclosed in brackets [ ]
- Spaces or commas separate elements in the same row
- Semicolons (;) or line breaks separate rows

Matrix Example

Matrix Definition

```
A = [1 2 3 4; 5 6 7 8; 9 10 11 12]
```

Equivalent Syntax

```
A=[1,2,3,4;5,6,7,8;9,10,11,12]
A=[1 2 3 4; 5 6 7 8; 9 10 11 12]
A=[[1;5;9],[2;6;10],[3;7;11],[4;8;12]]
```

Important Rule

The number of elements in each row must be the same.

Error Example

```
X = [1 2; 3 4 5]
```

Error: Dimensions of matrices being concatenated are not consistent.

Creating Matrices from Vectors

A matrix can be generated from vectors as shown in the following examples:

```
>> x = 1:4                    % creation of vector x
```

$$x = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

```
>> y = 5:5:20                 % creation of vector y
```

$$y = \begin{bmatrix} 5 & 10 & 15 & 20 \end{bmatrix}$$

```
>> z = 4:4:16                 % creation of vector z
```

$$z = \begin{bmatrix} 4 & 8 & 12 & 16 \end{bmatrix}$$

Matrix Built from Row Vectors

```
>> A = [x ; y ; z]

% A is formed from row vectors x, y and z
```

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 10 & 15 & 20 \\ 4 & 8 & 12 & 16 \end{bmatrix}$$

Matrix Built from Column Vectors

```
>> B = [x'  y'  z']
```

```
% B is formed from column vectors x, y and z
```

$$B = \begin{bmatrix} 1 & 5 & 4 \\ 2 & 10 & 8 \\ 3 & 15 & 12 \\ 4 & 20 & 16 \end{bmatrix}$$

Repeated Row Vector

```
>> C = [x ; x]
```

```
% C is formed from the same row vector x repeated twice
```

$$C = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

2.2 Indexing and Access

Accessing the elements of a matrix is done using the general syntax:

```
matrix_name(row_positions,column_positions)
```

Positions may be:

- a single number
- a list of numbers (position vector)

Parentheses ( ) are used for access.Brackets [ ] are used only during creation.

Notes

- Accessing the element in row i and column j: `A(i,j)`
- Accessing the whole row number i: `A(i,:)`
- Accessing the whole column number j: `A(:,j)`

Examples

```
>> A=[1,2,3,4;5,6,7,8;9,10,11,12]
```

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

```
>> A(2,3)
```

$$ans = 7$$

```
>> A(1,:)
```

$$ans = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

Examples

```
>> A(:,2)
```

$$ans = \begin{bmatrix} 2 \\ 6 \\ 10 \end{bmatrix}$$

More Matrix Indexing Examples

```
>> A(2:3,:)    % all elements of rows 2 and 3
```

$$ans = \begin{bmatrix} 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

```
>> A(1:2,3:4)    % upper-right 2x2 submatrix
```

$$ans = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}$$

```
>> A([1,3],[2,4])    % rows (1,3) and columns (2,4)
```

$$ans = \begin{bmatrix} 2 & 4 \\ 10 & 12 \end{bmatrix}$$

Deleting Rows and Columns

```
>> A(:,3)=[]     % delete the third column
```

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 5 & 6 & 8 \\ 9 & 10 & 12 \end{bmatrix}$$

```
>> A(2,:)=[]     % delete the second row
```

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 9 & 10 & 12 \end{bmatrix}$$

Adding Rows and Columns

```
>> A=[A,[0;0]]     % add a new column
```

$$A = \begin{bmatrix} 1 & 2 & 4 & 0 \\ 9 & 10 & 12 & 0 \end{bmatrix}$$

```
>> A=[A;[1,1,1,1]]     % add a new row
```

$$A = \begin{bmatrix} 1 & 2 & 4 & 0 \\ 9 & 10 & 12 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

The size Function

The function `size` returns the dimensions of a matrix.
For a matrix A of size $m \times n$, the result is a vector with two components:

$$[m, n]$$

```
>> d=size(A)
```

$$d = \begin{bmatrix} 3 & 4 \end{bmatrix}$$

Getting Dimensions Separately

To obtain dimensions separately:

```
>> d1=size(A,1)    % number of rows
```

$$d1 = 3$$

```
>> d2=size(A,2)    % number of columns
```

$$d2 = 4$$

Automatic Matrix Generation

There are functions that automatically generate special matrices:

Function	**Meaning**
`zeros(n)`	$n \times n$ matrix of zeros
`zeros(m,n)`	$m \times n$ matrix of zeros
`ones(n)`	$n \times n$ matrix of ones
`ones(m,n)`	$m \times n$ matrix of ones
`eye(n)`	$n \times n$ identity matrix
`magic(n)`	$n \times n$ magic square
`rand(m,n)`	$m \times n$ random matrix

Basic Matrix Operations

Operation	Meaning
$+$	Addition
$-$	Subtraction
$.*$	Element-wise multiplication
$./$	Element-wise division
$.\backslash$	Element-wise left division
$.^{}$	Element-wise power
$*$	Matrix multiplication
$/$	Matrix right division ($A/B = AB^{-1}$)

Remark: Element-wise operations on matrices are the same as for vectors, provided both matrices have the same dimensions.

Examples of Matrix Operations

```
>> A=ones(2,3)
```

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

```
>> B=zeros(3,2)
```

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

```
>> B=B+3
```

$$B = \begin{bmatrix} 3 & 3 \\ 3 & 3 \\ 3 & 3 \end{bmatrix}$$

Matrix Products and Modifications

```
>> A*B
```

$$ans = \begin{bmatrix} 9 & 9 \\ 9 & 9 \end{bmatrix}$$

```
>> B=[B,[3 3 3]']     % or B(:,3)=[3 3 3]'
```

$$B = \begin{bmatrix} 3 & 3 & 3 \\ 3 & 3 & 3 \\ 3 & 3 & 3 \end{bmatrix}$$

```
>> B=B(1:2,:)     % or B(3,:)=[]
```

$$B = \begin{bmatrix} 3 & 3 & 3 \\ 3 & 3 & 3 \end{bmatrix}$$

Element-wise and Matrix Multiplication

```
>> A=A*2
```

$$A = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix}$$

```
>> A.*B
```

$$ans = \begin{bmatrix} 6 & 6 & 6 \\ 6 & 6 & 6 \end{bmatrix}$$

```
>> A*eye(3)
```

$$ans = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix}$$

Useful Functions for Matrix Processing

Function	Utility	Example
det	Computes the determinant of a matrix	>>A=[1,2;3,4]; >>det(A) ans=-2
inv	Computes the inverse of a matrix	>>inv(A) ans= -2.0000 1.0000 1.5000 -0.50000
rank	Computes the rank of a matrix	>>rank(A) ans=2

More Useful Matrix Functions

Function	Utility	Example
trace	Computes the trace of a matrix	>>trace(A) ans=5
eig	Computes the eigenvalues of a matrix	>>eig(A) ans= -0.3723 5.3723
dot	Computes the dot product of to vectors	>>u=[-1,5,3]; v=[2,-2,1]; >>dot(u,v) ans=-9

Diagonal Functions

Function	Utility	Example
norm	Computes the norm of a vector	>norm(u) ans=3
cross	Computes the cross product of two vectors	>>cross(u,v) ans= -11 -7 8
diag	Returns the diagonal of a matrix	>>diag(A) ans= 1 4
diag(V)	Creates a matrix with vector V on the diagonal and zeros elsewhere	>>V=[-5,1,3] >>diag(V) ans= -5 0 0 0 1 0 0 0 3

Triangular functions

Function	Utility	Example
tril	Returns the lower triangular part	>>B=[1,2,3;4,5,6;7,8,9] B = 1 2 3 4 5 6 7 8 9 >>tril(B) ans = 1 0 0 4 5 0 7 8 9

Triangular functions

Function	Utility	Example
tril	Returns the lower triangular part	>>tril(B,-1) ans = 0 0 0 4 0 0 7 8 0 >>tril(B,-2) ans = 0 0 0 0 0 0 7 0 0

Triangular functions

Function	Utility	Example
triu	Returns the upper triangular part	>>triu(B) ans = 1 2 3 0 5 6 0 0 9

Triangular functions

Function	Utility	Example
triu	Returns the upper triangular part	>>triu(B,-1) ans = 1 2 3 4 5 6 0 8 9 >>triu(B,1) ans = 0 2 3 0 0 6 0 0 0

Matrix Comparison

The comparison of vectors and matrices differs slightly from scalars. Hence the usefulness of the two functions:

- `isequal`
- `isempty`

which provide a concise answer for comparison.

Comparison Functions

Function	Description
isequal	Tests whether two (or more) matrices are equal (having the same elements everywhere). Returns 1 if true, 0 otherwise.
isempty	Tests whether a matrix is empty (contains no elements). Returns 1 if true, 0 otherwise.

Example

MATLAB

```
>> A = [5,2;-1,3] % Create matrix A
A = 5 2 -1 3
>> B = [5,1;0,3] % Create matrix B
B = 5 1 0 3
```

Comparison Results

MATLAB

```
>> A==B % Test if A = B (1 or 0 depending on position)
ans = 1 0 0 1
>> isequal(A,B) % Test if A and B are exactly equal
ans = 0
>> C=[] % Create empty matrix C
>> isempty(C) % Test if C is empty (true = 1)
ans = 1
>> isempty(A) % Test if A is empty (false = 0)
ans = 0
```

Exercise 1: Basic Matrix Creation

Use the commands `zeros`, `ones`, and `eye` to create the following arrays:

- **Matrix a (Zeros):**

$$a = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Matrix b (Identity):**

$$b = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Matrix c (Ones):**

$$c = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Exercise 2: Matrix Indexing and Extraction

First, create the following matrix **A**:

$$A = \begin{bmatrix} 6 & 43 & 2 & 11 & 87 \\ 12 & 6 & 34 & 0 & 5 \\ 34 & 18 & 7 & 41 & 9 \end{bmatrix}$$

Use matrix **A** to perform the following tasks:

- **va**: Create a 5-row column vector containing the elements of the **second row** of A.
- **vb**: Create a 3-element column vector containing the elements of the **fourth column** of A.
- **vc**: Create a 10-element column vector containing the elements of the **first and second rows** of A.
- **vd**: Create a 6-element column vector containing the elements of the **first and fifth columns** of A.